

POCO4A

PROGRAMAÇÃO ORIENTADA

A OBJETOS

Aula 01 - Introdução à POO

Prof. Rafael G. **Mantovani**

Roteiro



- 1 Introdução**
- 2 Paradigmas de Programação: procedural vs OO**
- 3 Conceitos Básicos**
- 4 Exemplos: C++ vs Python**
- 5 Exercícios**
- 6 Referências**

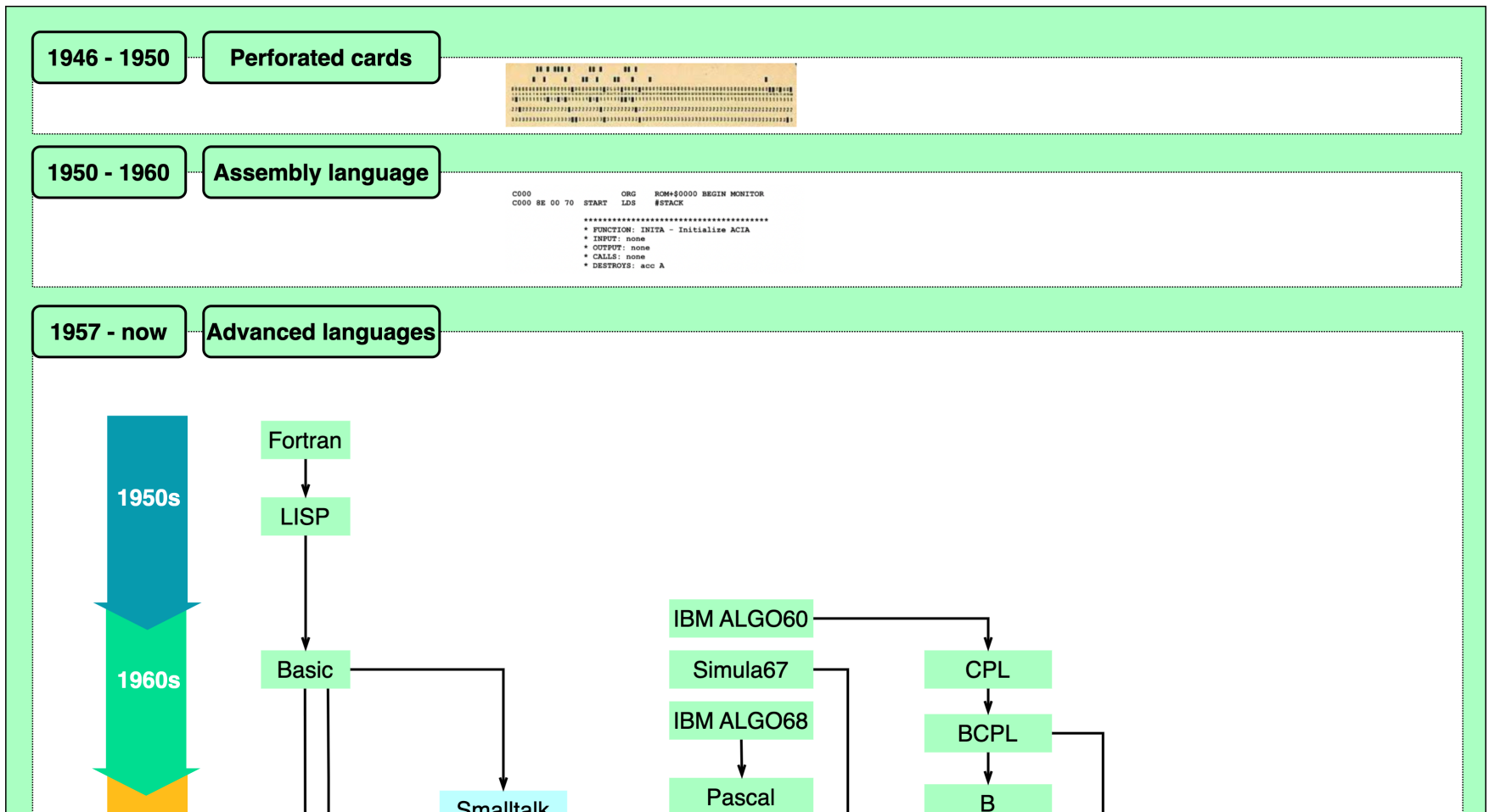
Roteiro

- 1 Introdução**
- 2 Paradigmas de Programação: procedural vs OO**
- 3 Conceitos Básicos**
- 4 Exemplos: C++ vs Python**
- 5 Exercícios**
- 6 Referências**

Introdução

C, C++, Java, Javascript, Typescript, Golang, Rust - A Brief History of Programming Languages

Fonte:  blog.bytebytego.com



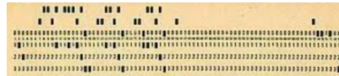
Introdução

**C, C++, Java, Javascript, Typescript, Golang, Rust
- A Brief History of Programming Languages**

Fonte:  blog.bytebytego.com

1946 - 1950

Perforated cards



- Cartões perfurados foram a primeira geração de linguagens de programação

Introdução

- Linguagens de Assembly, orientadas às máquinas, compõem a segunda geração de linguagens de programação

1950 - 1960

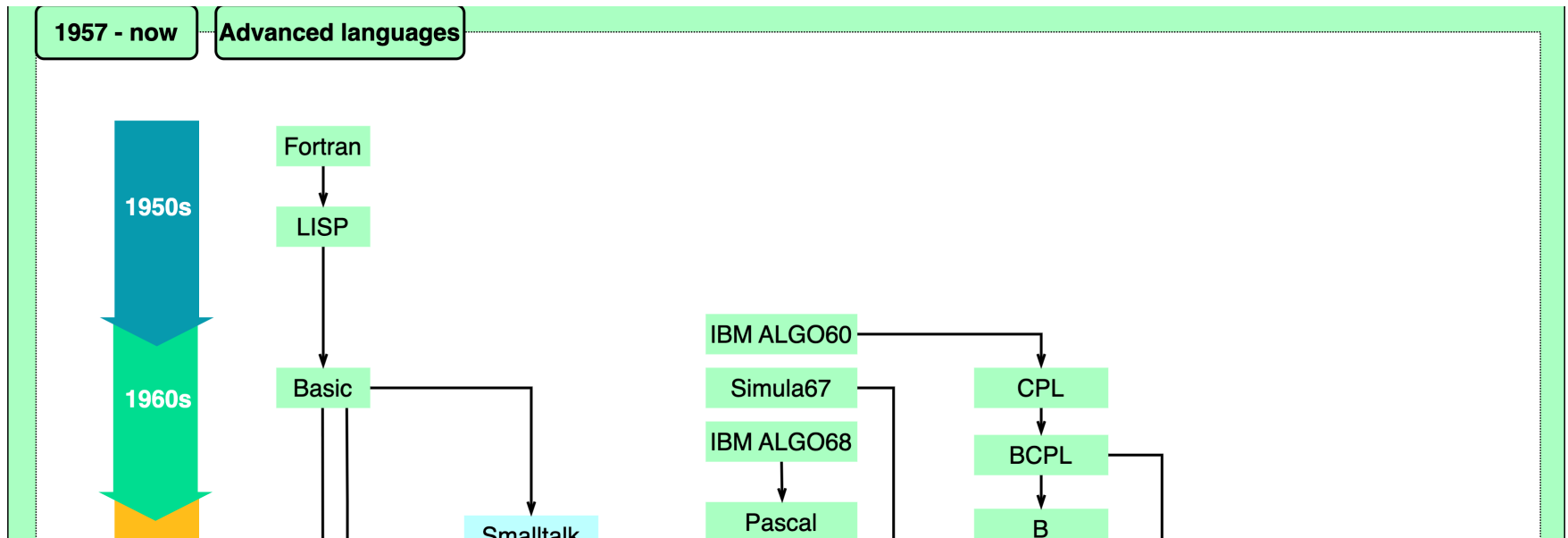
Assembly language

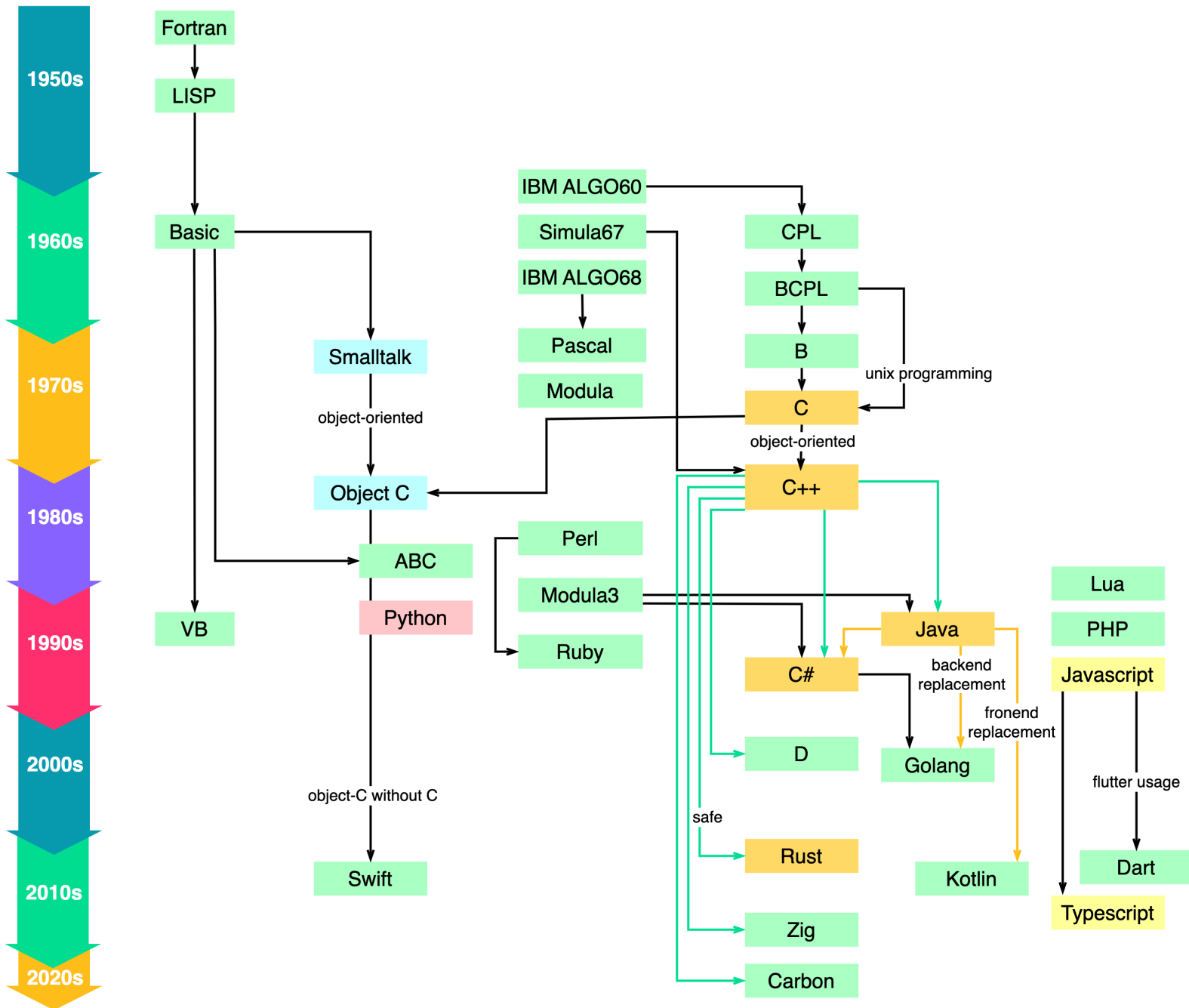
```
C000          ORG     ROM+0000 BEGIN MONITOR
C000 8E 00 70  START  LDS     #STACK

*****
* FUNCTION: INITA - Initialize ACIA
* INPUT: none
* OUTPUT: none
* CALLS: none
* DESTROY: acc A
```

Introdução

- Terceira-geração (1957 -) é composta pelas linguagens de programação orientadas por humanos





Introdução

Objetivos da aula:

- Compreender o que é POO
- Apresentar conceitos de **Classe** e **Objeto**
- Instanciar exemplos com Python/C++

Roteiro



- 1 Introdução
- 2 Paradigmas de Programação: procedural vs OO
- 3 Conceitos Básicos
- 4 Exemplos: C++ vs Python
- 5 Exercícios
- 6 Referências

Paradigmas



- Diferentes Linguagens de Programação:

Paradigmas

- Diferentes Linguagens de Programação:

C, **C++**, C#, Cobol, Fortran, **Java**, **JavaScript**, Lisp, Rust, **PHP**,
Prolog, **Python**, Ruby, Smalltalk, Visual Basic, ...

Paradigmas

- Diferentes Linguagens de Programação:

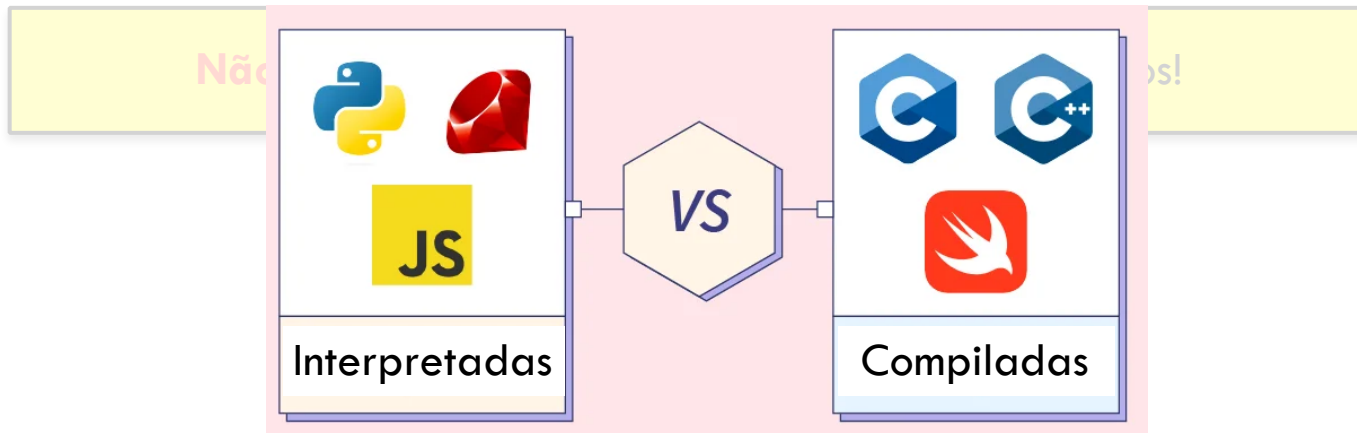
C, C++, C#, Cobol, Fortran, Java, JavaScript, Lisp, Rust, PHP, Prolog, Python, Ruby, Smalltalk, Visual Basic, ...

Não existe linguagem ideal para todos os cenários!

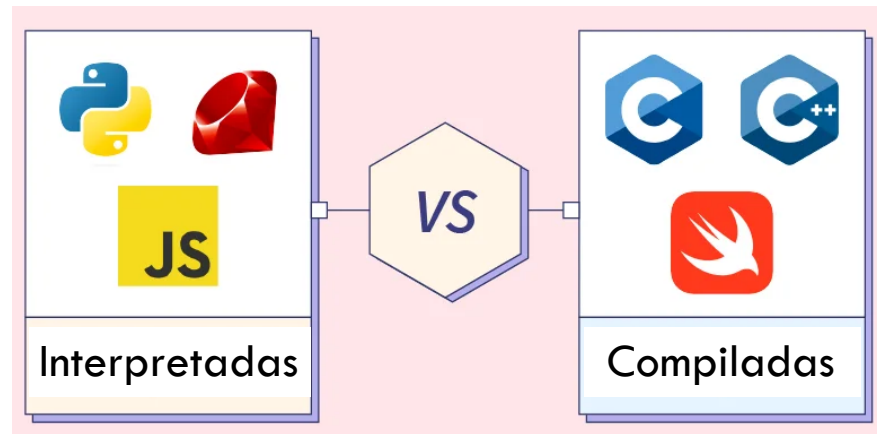
Paradigmas

- Diferentes Linguagens de Programação:

C, **C++**, C#, Cobol, Fortran, **Java**, **JavaScript**, Lisp, Rust, **PHP**,
Prolog, **Python**, Ruby, Smalltalk, Visual Basic, ...

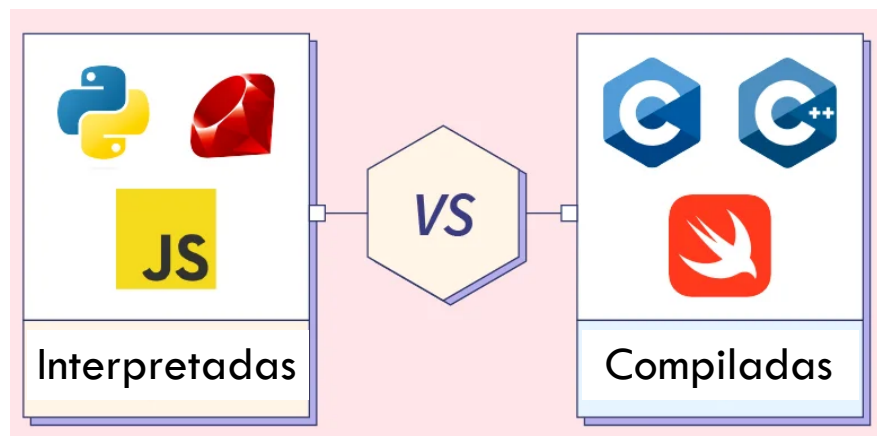


Paradigmas



Vantagens

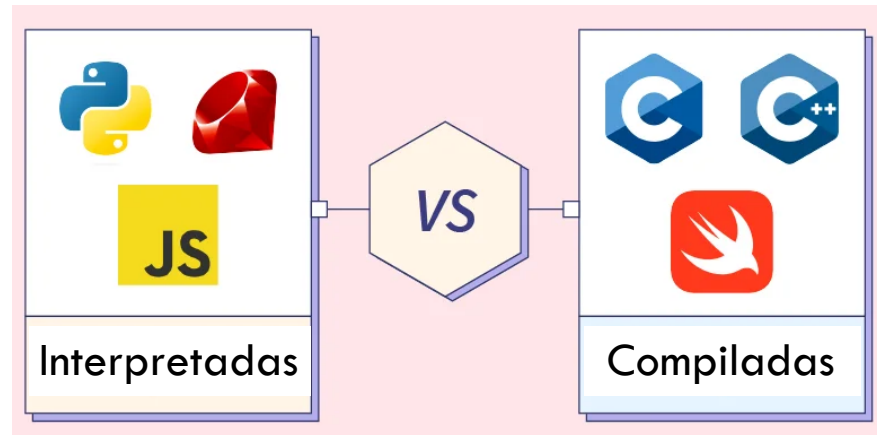
Paradigmas



Vantagens

- Testar o programa é mais fácil, porque é executado linha por linha
- Edição mais fácil e rápida de código
- Uma vez gerado o executável, não precisa compilar novamente
- Protege o código fonte de manipulação de outros agentes/softwares
- Programas executam a plena capacidade porque o código fonte não precisa ser compilado durante a execução

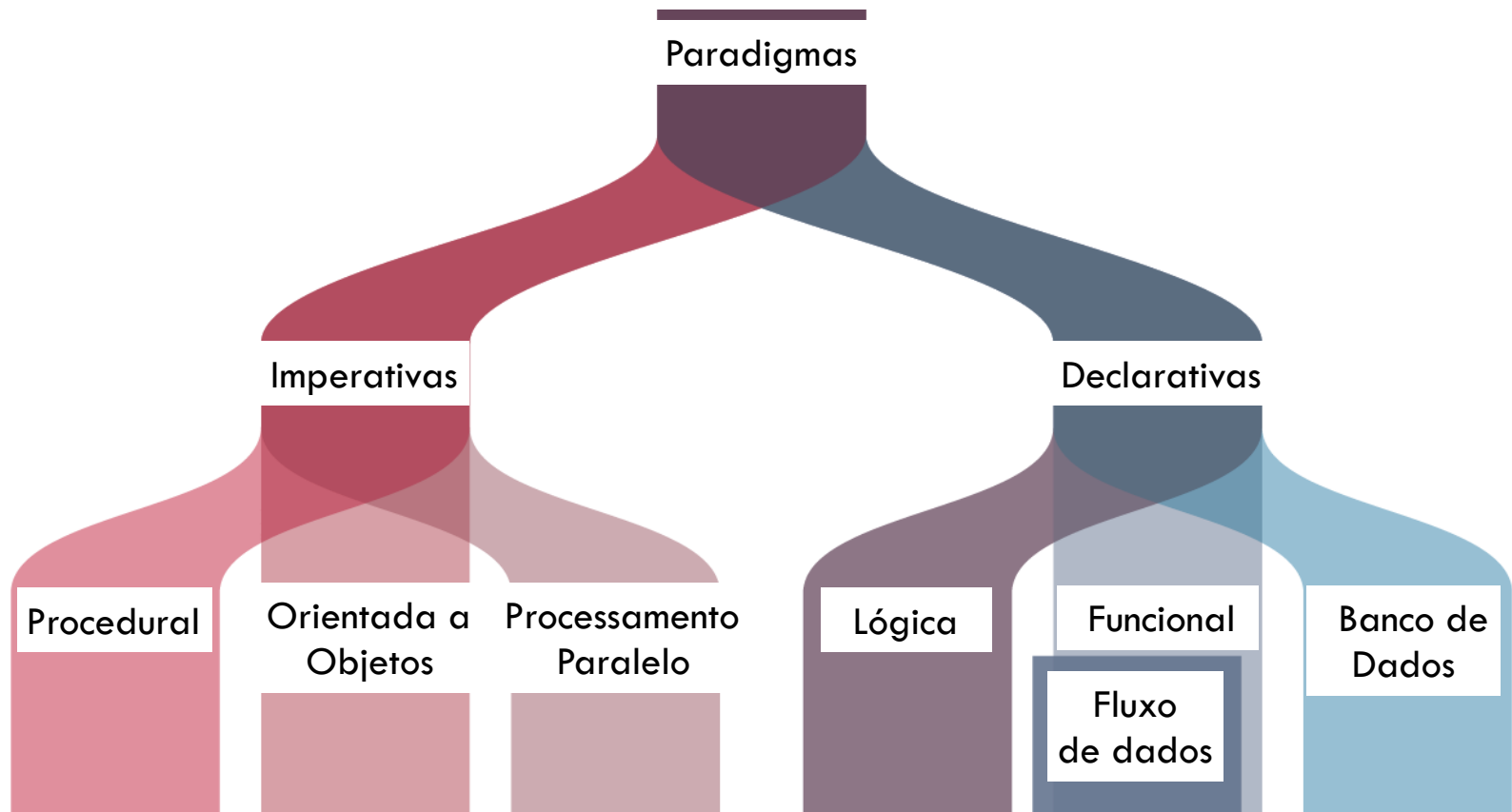
Paradigmas



Desvantagens

- Por ser traduzido durante a execução, impacta na funcionalidade do software
- Código-fonte é acessível e pode levar a problemas se disponibilizado assim
- Dificuldade em acessar o código-fonte novamente depois da compilação

Paradigmas



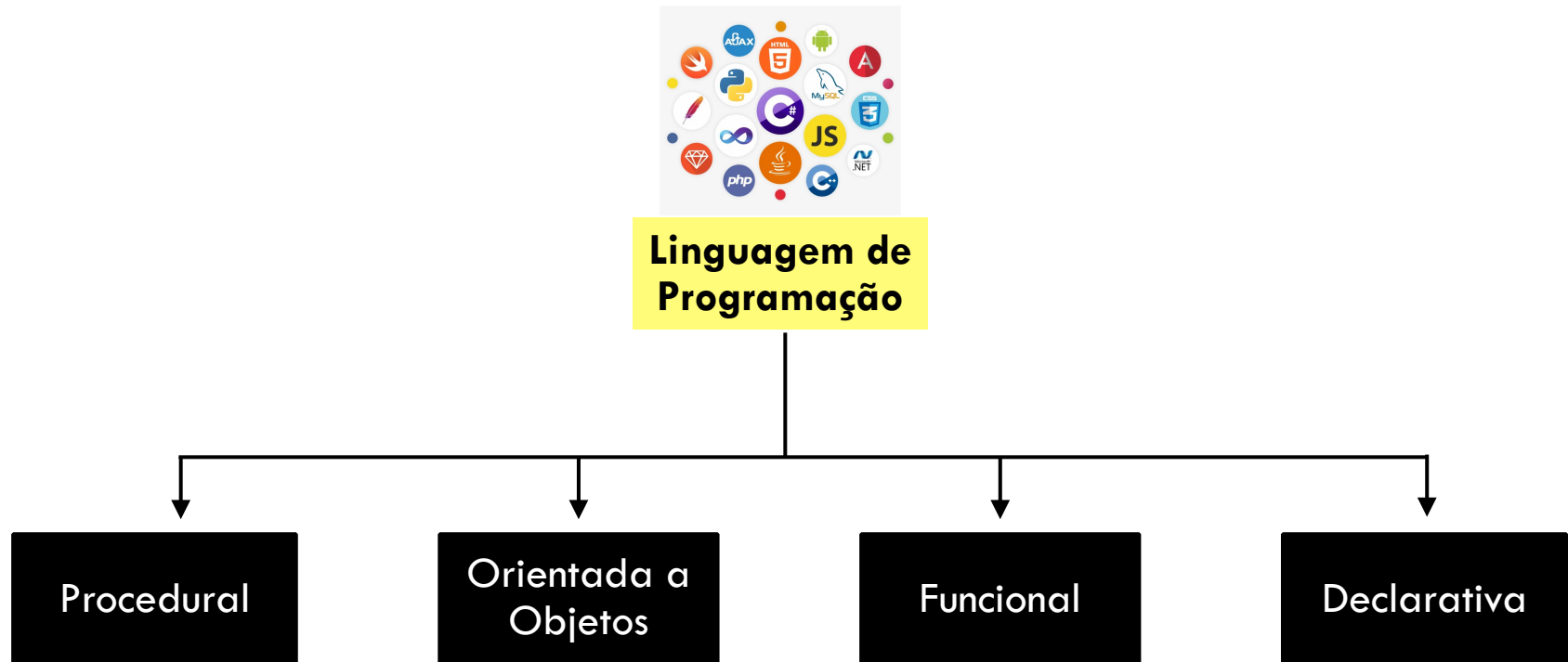
Fonte: [link](#)

Paradigmas



**Linguagem de
Programação**

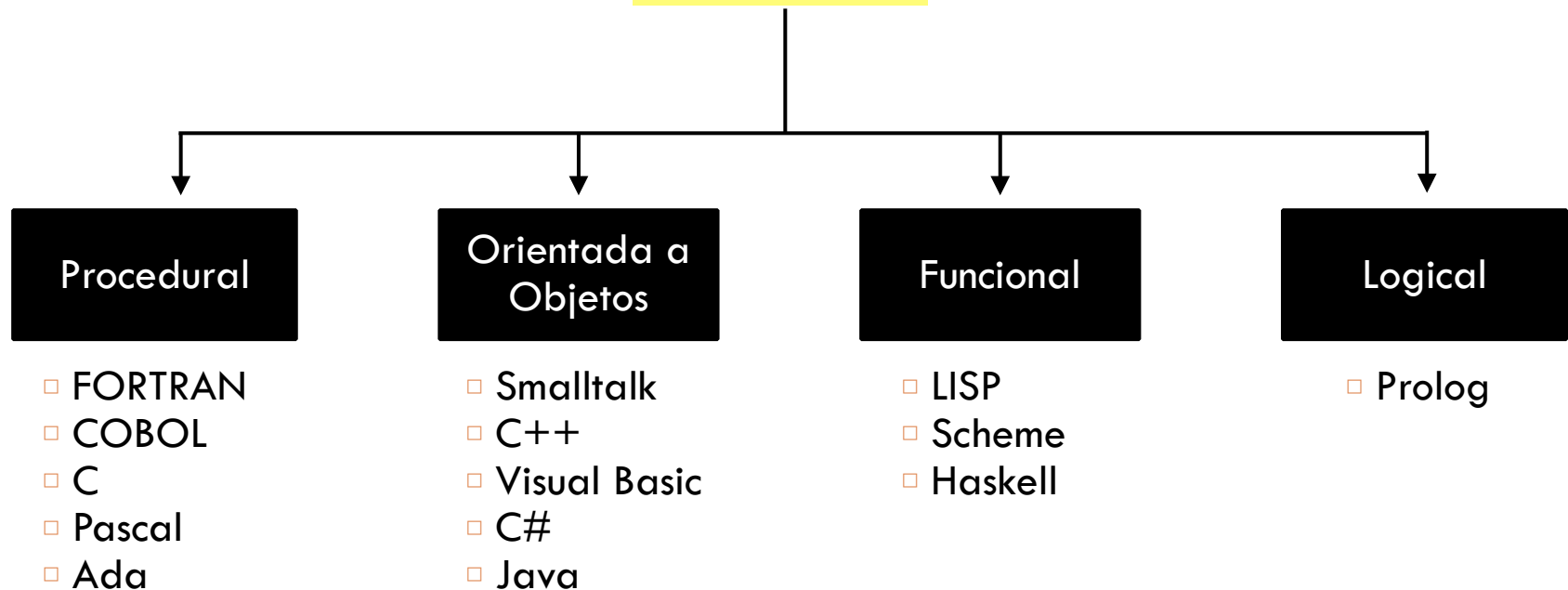
Paradigmas



Paradigmas



Linguagem de Programação



Paradigmas

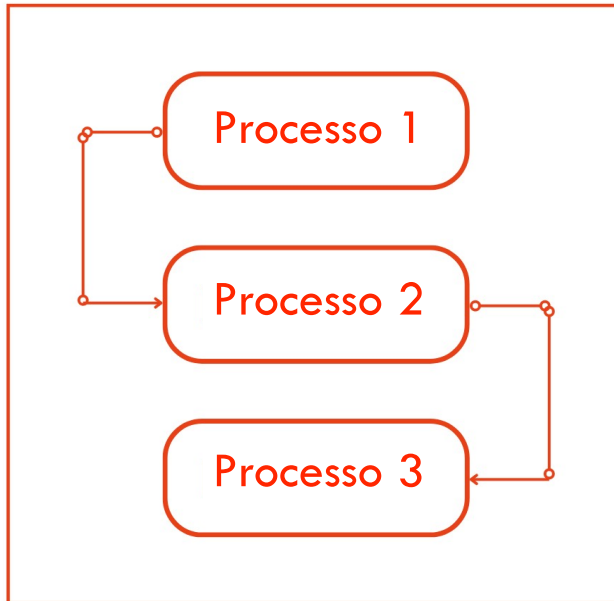


Procedural

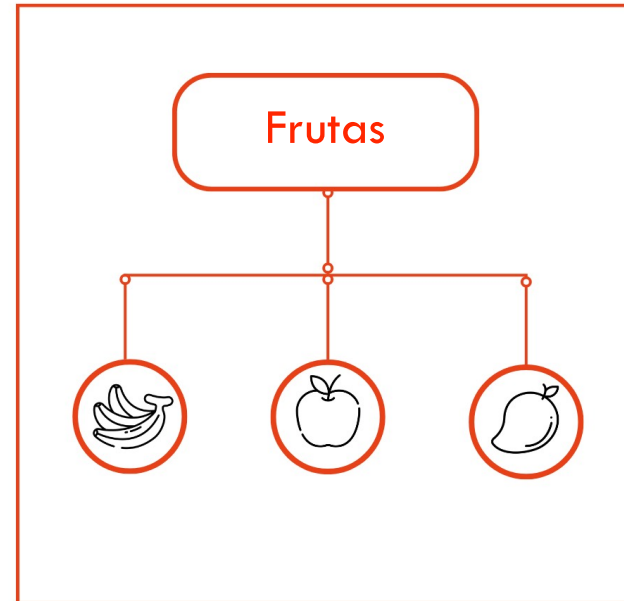
Orientado a Objetos

Paradigmas

Procedural

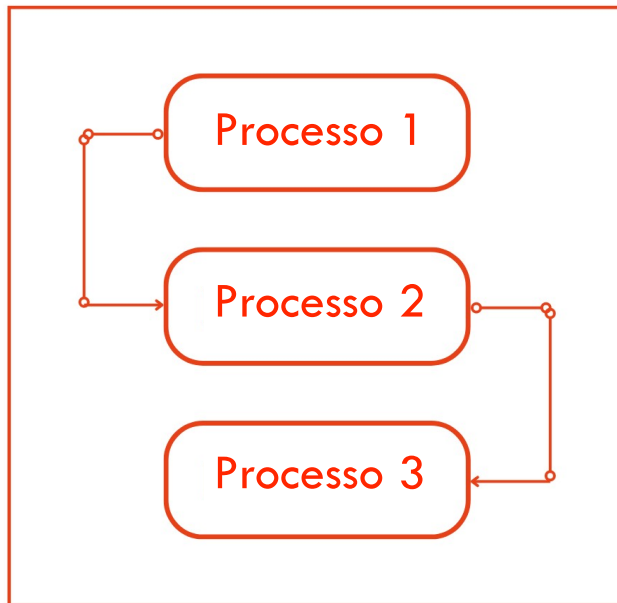


Orientado a Objetos



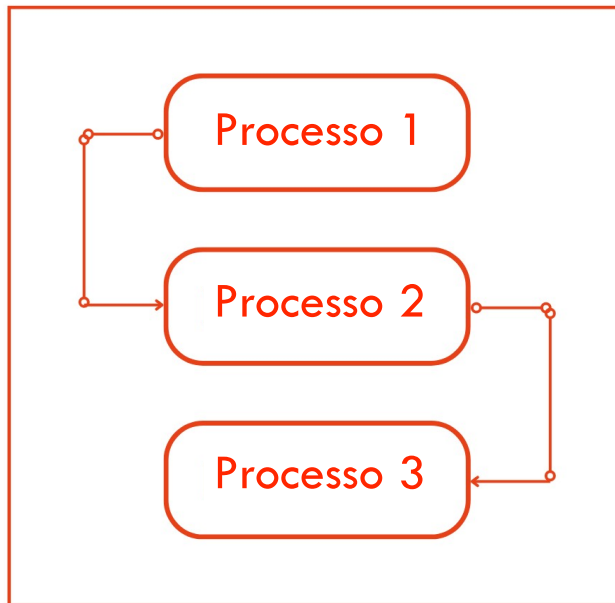
Paradigmas

Procedural



Paradigmas

Procedural



Código dividido em **funções** ou módulos

Função **main** faz a comunicação com os outros métodos

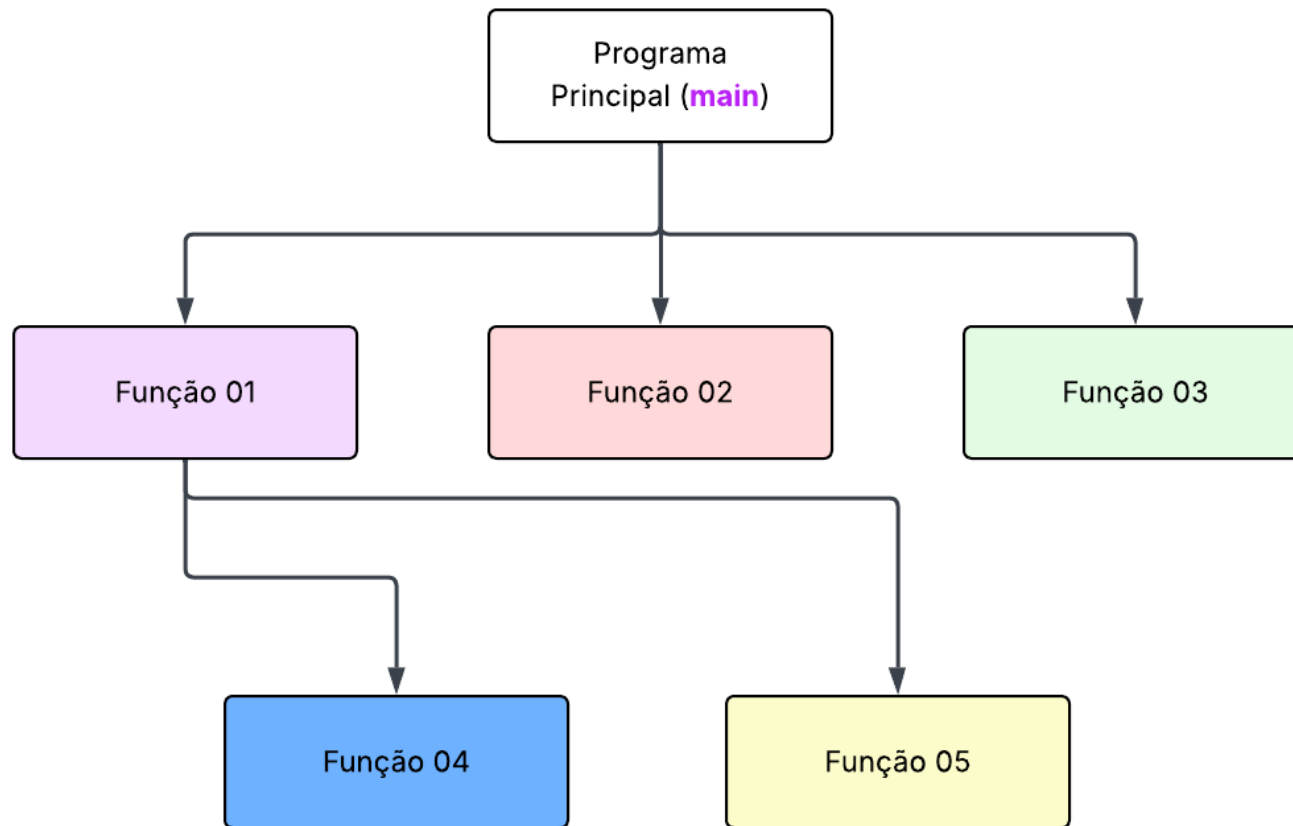
Difícil de reutilizar código

Paradigmas



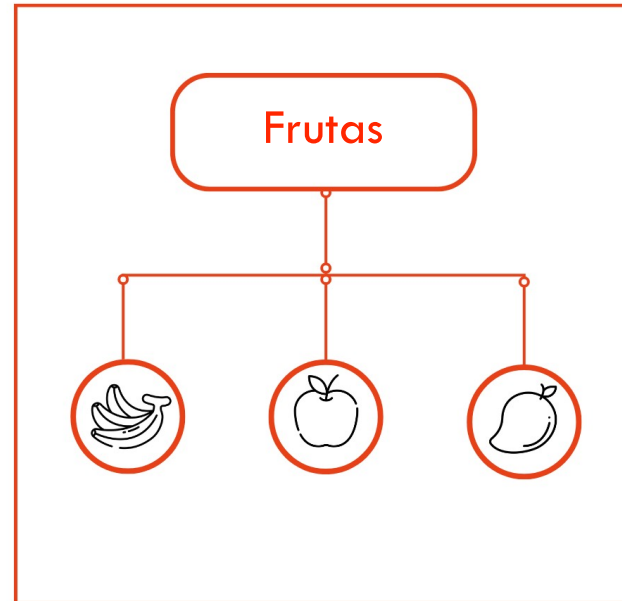
Programa
Principal (**main**)

Paradigmas



Paradigmas

Orientado a Objetos



Paradigmas

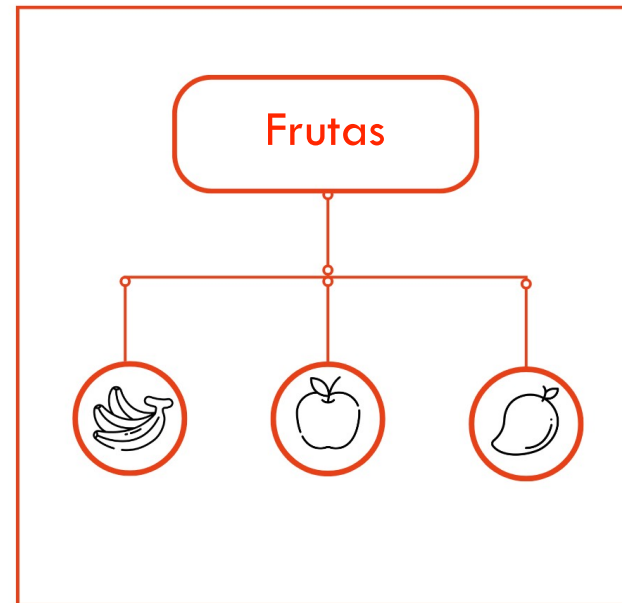
Abordagem baseada nos **dados**

Programa é um conjunto de **objetos** interagindo entre si

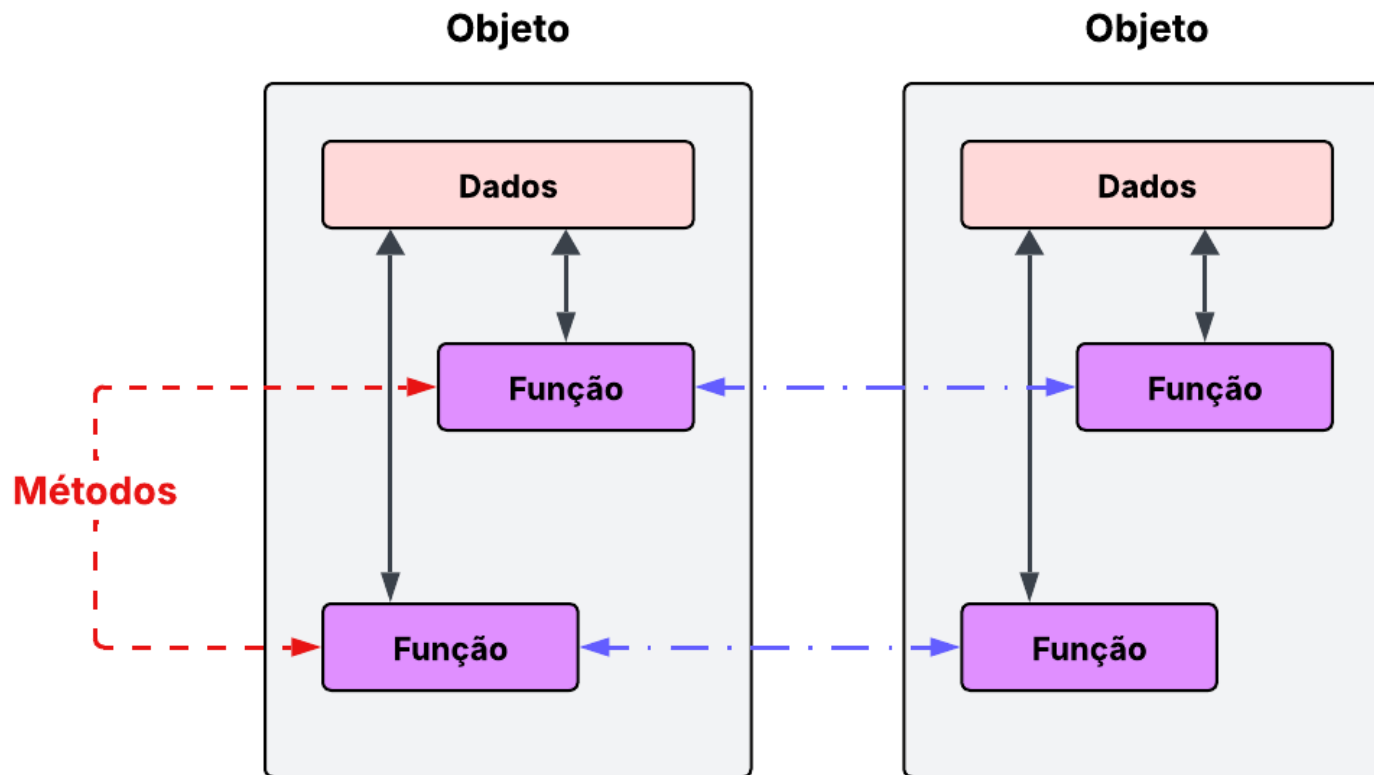
Segurança é garantida por meio dos especificados de acesso

Reuso de código facilitado pelo conceito de **herança**

Orientado a Objetos



Paradigmas



Roteiro

- 1 Introdução
- 2 Paradigmas de Programação: procedural vs OO
- 3 Conceitos Básicos
- 4 Exemplos: C++ vs Python
- 5 Exercícios
- 6 Referências

POO

POO

POO foi desenvolvida para dar suporte a vários modos de conceitualização e expressão de software

Baseada na modelagem do par (**dados, operações**) como um elemento **único**, e não elementos separados

POO



Exemplo: Imagem Digital

POO

Exemplo: Imagem Digital



POO

Exemplo: Imagem Digital → **Objeto**



POO

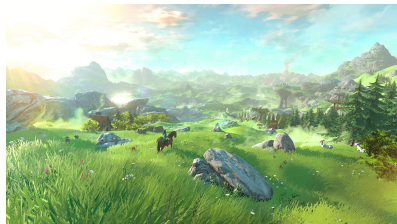


Imagem Digital

POO



Imagem Digital

Dados

- △ Resolução
- △ Nome/Extensão
- △ Canais de cores
- △ ...

POO



Imagem Digital

Dados

Operações

- Resolução
- Nome/Extensão
- Canais de cores
- ...

- Escalar
- Rotacionar
- Recortar
- Mostrar
- ...

POO



Imagem Digital

Dados

- △ Resolução
- △ Nome/Extensão
- △ Canais de cores
- △ ...

Operações

- ⊞ Rotacionar
- ⊞ Escalar
- ⊞ Recortar
- ⊞ Mostrar
- ⊞ ...



Imagem Digital

Imagem Digital
+ resolução + nome + extensão + canal de cor + ...
+ rotacionar() + escalar (novaEscala) + mostrar() + recortar () + ...

POO

Objetos são entidades que interagem durante a execução de um programa, e que são organizadas em **Classes**, baseados em suas características comuns



Imagem Digital

+ nome
+ extensão
+ canal de cor
+ ...
+ rotacionar ()
+ escalar (novaEscala)
+ mostrar ()
+ recortar ()
+ ...

POO

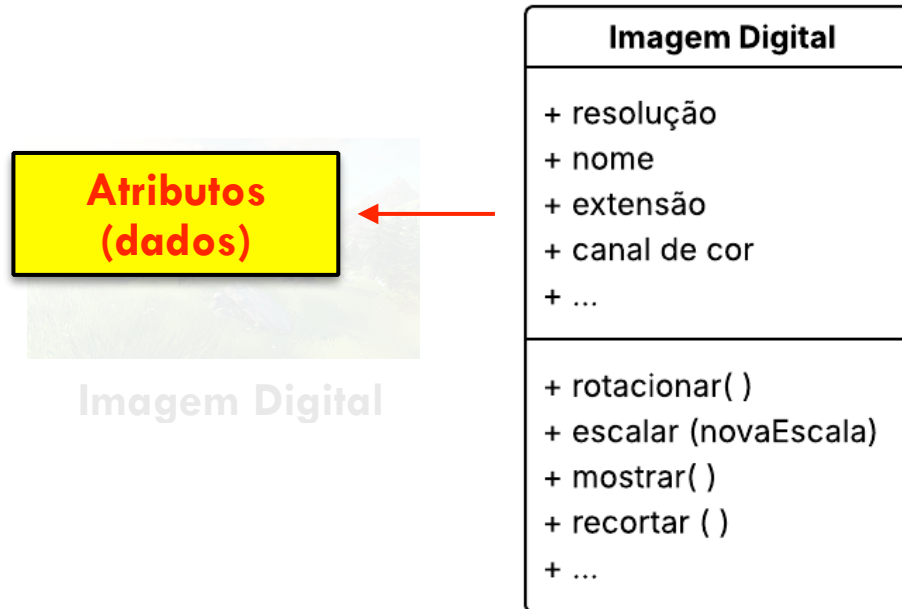
Objetos são entidades que interagem durante a execução de um programa, e que são organizadas em **Classes**, baseados em suas características comuns

Objetos de uma mesma classe compartilham uma mesma codificação de representação de dados e possuem um mesmo conjunto de operações

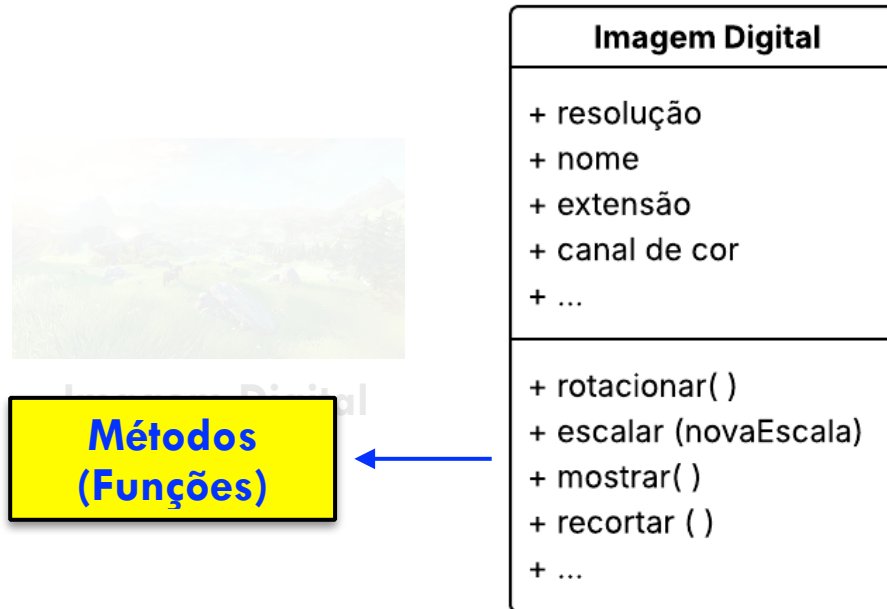
+ nome
+ extensão

+ mostrar ()
+ recortar ()
+ ...

POO



POO

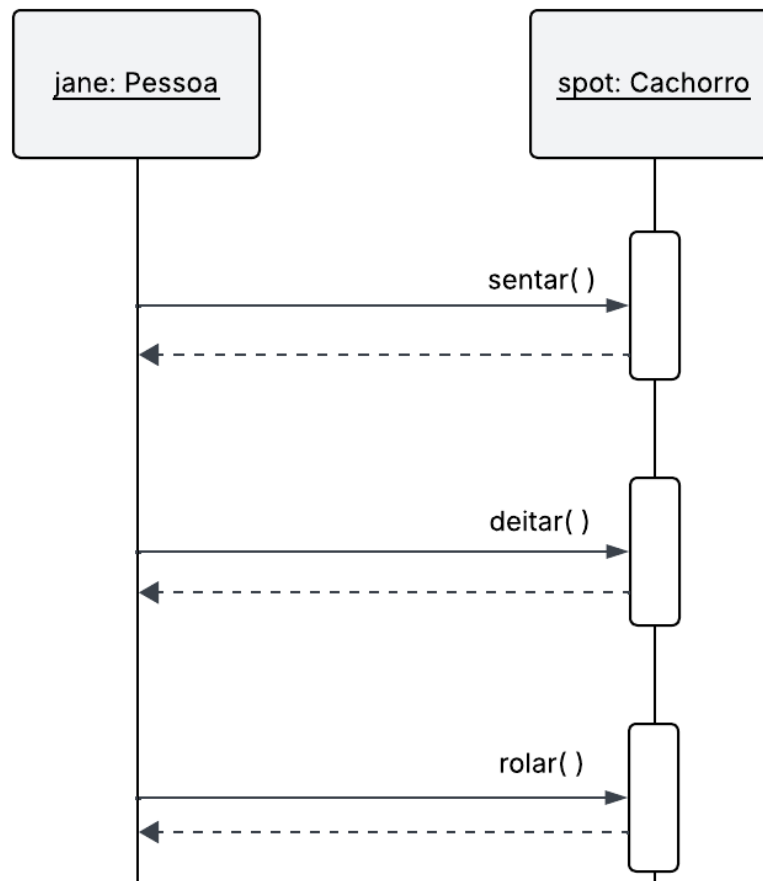


POO

Exemplo: Cão obediente

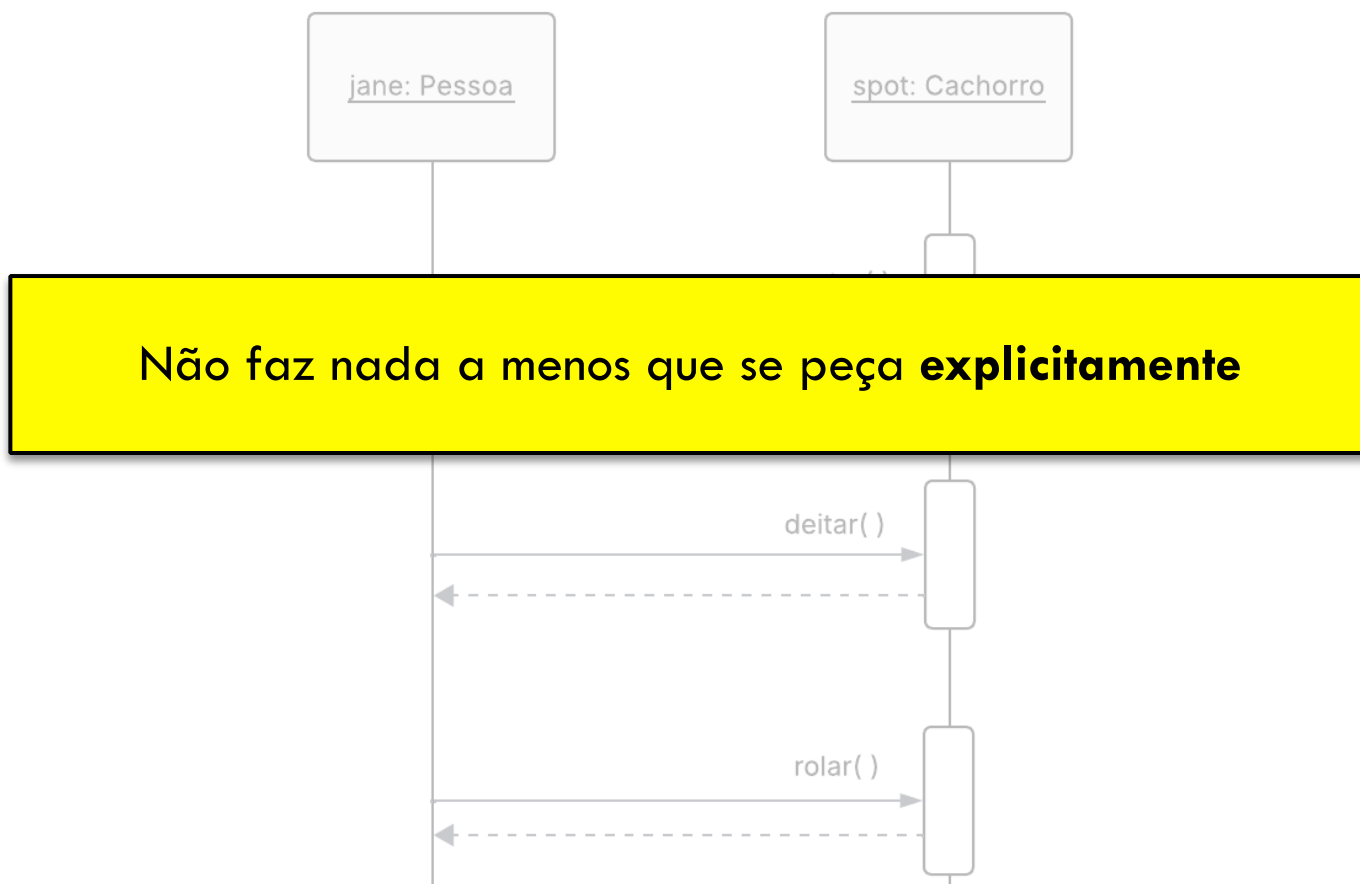
POO

Exemplo: Cão obediente



POO

Exemplo: Cão obediente



POO

Exemplo: Televisão

POO



Exemplo: Televisão

Como determinar se alguém viu TV enquanto você
estava fora de casa?

POO

Exemplo: Televisão

TV
marca modelo estaLigada volume estaMutada canal canalAnterior
ligar() desligar() aumentarVolume() diminuirVolume() mutar() desmutar() proximoCanal() voltarCanal() acessarCanal(numeroCanal) ...

Roteiro

- 1 Introdução
- 2 Paradigmas de Programação: procedural vs OO
- 3 Conceitos Básicos
- 4 Exemplos: C++ vs Python
- 5 Exercícios
- 6 Referências

Exemplos



Python

Tipagem **dinâmica**: não declara tipos

Indentação define blocos

self explícito como primeiro parâmetro

__init__ como construtor

Tudo é público por padrão (convenção
_privado)

Parâmetros com valores padrão: def
__init__(self, hp=100)



C++

Tipagem **estática**: deve declarar tipos (int, string, etc)

Chaves {} definem blocos

this implícito, raramente usado

Função com nome da classe

Controle explícito: private, protected, public

Valores padrão: Jogador(string n, int h = 100)

Exemplo 01: classe vazia



Python

```
1. class ImagemDigital:
2.     # Classe vazia válida
3.     pass
4.
5. # Função Principal
6. if __name__ == "__main__":
7.     foto = ImagemDigital()
8.     print(type(foto))
9.
10.
10.
```



C++

```
#include <iostream>
#include <typeinfo>

class ImagemDigital {
    # Classe vazia válida
};

# Função Principal
int main(int argc, char const* argv[])
{
    ImagemDigital foto;
    cout << typeid(myObject).name();
    return 0;
}
```

Exemplo 01: classe vazia



Python

```
1. class ImagemDigital:
2.     # Classe vazia válida
3.     pass
4.
5. # Função Principal
6. if __name__ == "__main__":
7.     foto = ImagemDigital()
8.     print(type(foto))
9.
10.
10.
```



C++

```
#include <iostream>
#include <typeinfo>

class ImagemDigital {
    # Classe vazia válida
};

# Função Principal
int main(int argc, char const* argv[])
{
    ImagemDigital foto;
    cout << typeid(myObject).name();
    return 0;
}
```



Python: 'pass' para classe vazia | **C++:** corpo vazio { } também válido

Exemplo 2: adicionando atributos



C++

Exemplo 2: adicionando atributos



C++

```
1. #include <iostream>
2. #include <string>
3.
4. class ImagemDigital {
5.     // Adicionando atributos
6.     public:
7.         int largura;
8.         int altura;
9.         int **imagem;
10.        string nome
11.        string extensao;
12.
13.        //Construtor da Classe
14.        ImagemDigital(int l, int a, string n, string e) {
15.            largura = l; altura = a;
16.            nome = n; extensão = e;
17.        }
18.    };
```

Exemplo 2: adicionando atributos



C++

```
19. // Função Principal
20. int main(int argc, char const *argv[]) {
21.
22.     cout << "Criando objeto" << endl;
23.     ImagemDigital foto(256, 256, "teste", ".png");
24.
25.     cout << "Arquivo: " + foto.nome + foto.extensao << endl;
26.     cout << "Resolução: " << to_string(foto.altura) +
27.         " X " << to_string(foto.largura) << endl;
28.
29.     return 0;
30. }
```

Exemplo 3: adicionando métodos



C++

Exemplo 3: adicionando métodos



```
19. // ...
20. string CriaNome() {
21.     return(nome + extensao);
22. }
23.
24. string retornaResolucao() {
25.     return(to_string(altura) + " X " +
26.         to_string(largura));
27. }
28. };
```

Exemplo 3: adicionando métodos



C++

```
29. // Função Principal
30. int main(int argc, char const *argv[]) {
31.
32.     cout << "Criando objeto" << endl;
33.     ImagemDigital foto(256, 256, "teste", ".png");
34.
35.     cout << "Arquivo: " + foto.criaNome() << endl;
36.     cout << "Resolução: " + foto.retornaResolucao() << endl;
37.
38.     return 0;
39. }
```

Roteiro

- 1 Introdução
- 2 Paradigmas de Programação: procedural vs OO
- 3 Conceitos Básicos
- 4 Exemplos: C++ vs Python
- 5 Exercícios
- 6 Referências

Exercícios



HANDS ON :)))

Exercício 01

- Implemente uma classe de **ImagemDigital** em **Python**, adicionando atributos, métodos e faça testes de atribuição e acesso de valores em uma função principal.

Exercício 02



- Adicione novos métodos na classe **ImagemDigital**

Exercício 03

- Implemente classes em **C++** e **Python** para representação de uma **TV**

TV
marca modelo estaLigada volume estaMutada canal canalAnterior
ligar() desligar() aumentarVolume() diminuirVolume() mutar() desmutar() proximoCanal() voltarCanal() acessarCanal(numeroCanal) ...

Entregável Semanal

- **Proposta:**
 - Sistema de Jogo RPG (**Player + Inventory**)

Entregável Semanal

- **Proposta:**
 - Sistema de Jogo RPG (**Player + Inventory**)



Entregável Semanal

□ Proposta:

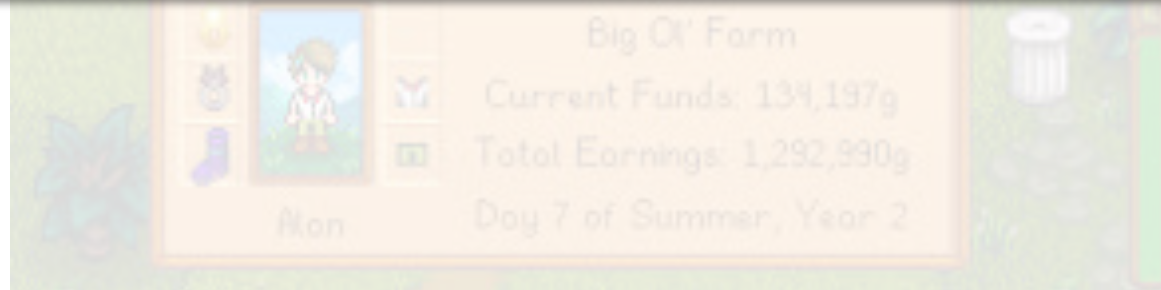
- Sistema de Jogo RPG (**Player** + **Inventory**)

Classe: Jogador/Player/Character

Atributos: nome, nível, HP, HP máximo

Métodos: exibirStatus(), estaVivo(), receberDano(), curar()

Linguagens: Python e C++



Roteiro

- 1 Introdução
- 2 Paradigmas de Programação: procedural vs OO
- 3 Conceitos Básicos
- 4 Exemplos: C++ vs Python
- 5 Exercícios
- 6 Referências

Referências sugeridas



[Deitel & Deitel, 2006]

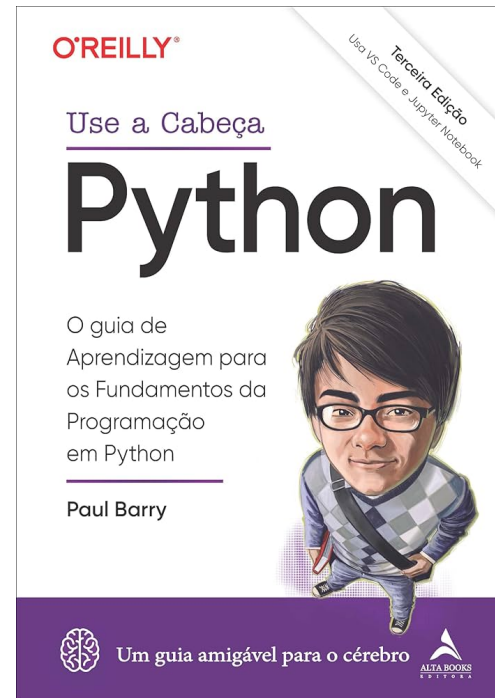


[Meyers, 2016]

Referências sugeridas



[Ziviani, 2007]



[Barry, 2025]



Obrigado :)

Prof. Rafael G. **Mantovani**
rafaelmantovani@utfpr.edu.br